COS720 Project - Insider Threat Detection System

Comprehensive Setup and Documentation Guide

---

1. SYSTEM OVERVIEW

Project Description

The Insider Threat Detection System is a machine learning-based solution designed to identify and classify potential insider threats within an organization. The system leverages a trained neural network model to analyze behavioral and access patterns, predicting the likelihood of malicious activity.

System Components

1. **Backend Service**: FastAPI-based REST API server
2. **Frontend Interface**: Svelte-based single-page application
3. **ML Model**: PyTorch neural network

Data Flow

1. User inputs features through the frontend interface
2. Features are sent to the backend API
3. Backend processes and scales the features
4. Model makes prediction
5. Results returned to frontend for visualization

---

2. SOFTWARE REQUIREMENTS AND DEPENDENCIES

System Requirements

- Operating System: Windows 10/11, macOS, or Linux
- Python: Version 3.9 or later (recommended: 3.11.x)
- Node.js: Version 18.0.0 or later (for frontend)
- RAM: Minimum 4GB (recommended: 8GB)
- Disk Space: 2GB for dependencies and models

Backend Dependencies

The backend uses Python with the following critical packages:

Package	Version
fastapi	0.136.1
joblib	1.5.1
pandas	3.0.3
shap	0.51.0
torch	2.7.1
uvicorn	0.47.0

python-multipart	0.0.29

Frontend Dependencies

The frontend uses Node.js with the following packages:

Package	Version
svelte	5.51.0
@sveltejs/kit	2.50.2
vite	7.3.1
tailwindcss	4.1.18
typescript	5.9.3

3. Installation Instructions

Backend

1. Navigate to the backend directory

```
cd Project/backend
```

2. Create and activate a virtual environment

```
# Create
python -m venv venv

# Activate — Windows
venv\Scripts\activate

# Activate — macOS/Linux
source venv/bin/activate
```

3. Install dependencies

```
pip install -r requirements.txt
```

Frontend

1. Navigate to the frontend directory

```
cd Project/frontend
```

2. Install dependencies

```
npm install
```

Key packages installed:

---

4. Deployment and Execution Instructions

Both the backend and frontend must be running simultaneously. Use two separate terminal windows or tabs. Ensure the `model` folder is present in order to load the model artefacts into the backend.

Terminal 1 — Backend

```
# Navigate and activate virtual environment
cd Project/backend

# Windows
venv\Scripts\activate
# macOS/Linux
source venv/bin/activate

# Start the server
python -m uvicorn main:app --reload
```

The API will be available at `http://localhost:8000`.

Terminal 2 — Frontend

```
cd Project/frontend
npm run dev
```

The application will be available at `http://localhost:5173`.

Verify the Setup

Once both servers are running, open `http://localhost:5173` in your browser.

ping the backend server to check if it is running

```
curl http://localhost:8000/ping
```

The response should be `{"message":"pong"}`

---

5. Source Code Documentation

Folder structure

```
Project/
├── SETUP.md
├── backend/
│   ├── main.py
│   ├── ThreatModels.py
│   ├── requirements.txt
│   ├── README.md
│   └── insider_threat_clean_dataset_test.csv
├── frontend/
│   ├── package.json
│   ├── svelte.config.js
│   ├── vite.config.ts
│   ├── src/
│   │   ├── app.html
│   │   ├── lib/
│   │   │   ├── assets/
│   │   │   │   └── favicon.svg
│   │   │   ├── models/
│   │   │   │   └── Predciction.ts
│   │   │   └── utils/
│   │   │       └── featureMetadata.ts
│   │   └── routes/
│   │       ├── +layout.svelte
│   │       ├── +page.svelte
│   │       └── layout.css
│   └── static/
│       └── robots.txt
└── model/
    ├── model.ipynb
    ├── model.joblib
    ├── model_weights.pth
    └── scaler.joblib
```

Backend

`main.py`

The entry point for the FastAPI application. Responsible for:

- Initialising the FastAPI app instance and configuring CORS middleware
- Defining API route handlers (endpoints) for receiving prediction requests
- Loading serialised model artefacts (`model.joblib` , `model_weights.pth` , `scaler.joblib`) at startup
- Passing incoming feature data to `ThreatModels.py` for inference and returning the result as a JSON response

Key endpoints:

Method	Path	Description
`GET`	`/ping`	Health check — confirms the API is running
`POST`	`/predict`	Accepts feature data and returns a threat prediction

`ThreatModels.py`

Contains the model inference logic, decoupled from the routing layer. Responsible for:

- Defining the data schema (Pydantic models) for validating incoming request payloads
- Preprocessing input features using the loaded scaler before inference
- Running forward passes through the trained model to generate predictions
- Optionally computing SHAP values for explainability alongside predictions

---

`requirements.txt`

Declares all Python dependencies required to run the backend. See Section 3 for the full package list.

Frontend

`src/routes/+layout.svelte`

The root layout component that wraps every page in the application. Defines shared structural elements such as navigation, headers, and global styles imported from `layout.css`.

`src/routes/+page.svelte`

The application's landing page. Renders the primary user interface for submitting employee feature data and displaying the resulting threat prediction returned by the backend API.

`src/routes/layout.css`

Global CSS styles scoped to the layout. Defines base typography, spacing, colour tokens, and utility classes shared across all pages.

`src/lib/models/Prediction.ts`

TypeScript type definitions and interfaces for the prediction data model. Defines the shape of:

- The request payload sent to `POST /predict`
- The response object returned by the backend, including the predicted class and confidence score

Ensures type safety across all API interactions in the frontend.

`src/lib/utils/featureMetadata.ts`

A utility module that holds metadata about the input features used by the model. Includes:

- Feature names and their expected data types
- Display labels used to render form fields in the UI
- Valid value ranges or categorical options where applicable

Acts as the single source of truth for feature definitions, keeping the UI and API payload in sync.

`src/app.html`

The root HTML shell for the SvelteKit application. Contains the `%sveltekit.head%` and `%sveltekit.body%` injection points used by the framework at build time.

`package.json`

Declares all Node.js dependencies and project scripts. Key scripts:

Script	Command	Description
Dev server	`npm run dev`	Starts Vite dev server with HMR
Production build	`npm run build`	Compiles and bundles the app
Preview	`npm run preview`	Serves the production build locally

Model

`model.ipynb`

A Jupyter notebook documenting the full model development pipeline:

1. Data loading and exploratory data analysis (EDA)
2. Feature engineering and selection
3. Data preprocessing and train/test splitting
4. Model training and hyperparameter tuning
5. Evaluation
6. Serialisation of the trained model and scaler to disk

---

6. Description of Trained Model Files

All trained model artefacts are located in the `model/` directory and are loaded by the backend at startup.

`model.joblib`

Property	Details
Format	joblib serialised object
Framework	scikit-learn
Purpose	Primary classification model for insider threat detection

Contains the serialised scikit-learn estimator (e.g. Random Forest, Gradient Boosting, or similar ensemble). This object encapsulates the trained parameters and is used to produce binary or probabilistic predictions from preprocessed input features. Loaded in `main.py` via `joblib.load()`.

`model_weights.pth`

Property	Details
Format	PyTorch state dictionary (`.pth`)
Framework	PyTorch (`torch`)
Purpose	Weights for a trained neural network component

Contains the saved state dictionary of a PyTorch neural network, serialised using `torch.save()`. Loaded at runtime via `torch.load()` into the corresponding model architecture defined in `ThreatModels.py`. Used independently or in conjunction with `model.joblib` depending on the inference pipeline.

`scaler.joblib`

Property	Details
Format	joblib serialised object
Framework	scikit-learn
Purpose	Feature normalisation / standardisation

Contains a fitted scikit-learn scaler (e.g. `StandardScaler` or `MinMaxScaler`) that transforms raw input features into the same numerical range seen during training. **Must be applied to all input data before inference.** Loaded in `main.py` via `joblib.load()` and called in `ThreatModels.py` prior to every prediction.

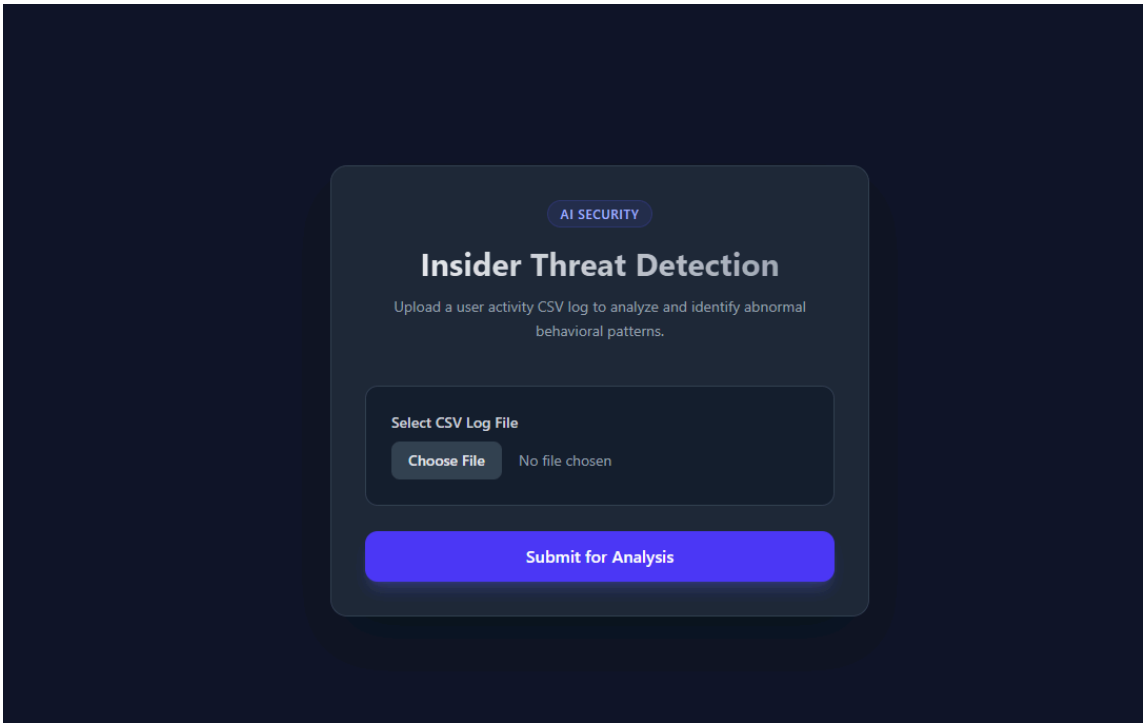
> ⚠️ *The scaler must not be retrained on new data without also retraining the model. Both artefacts are coupled — they were fitted on the same training set.*

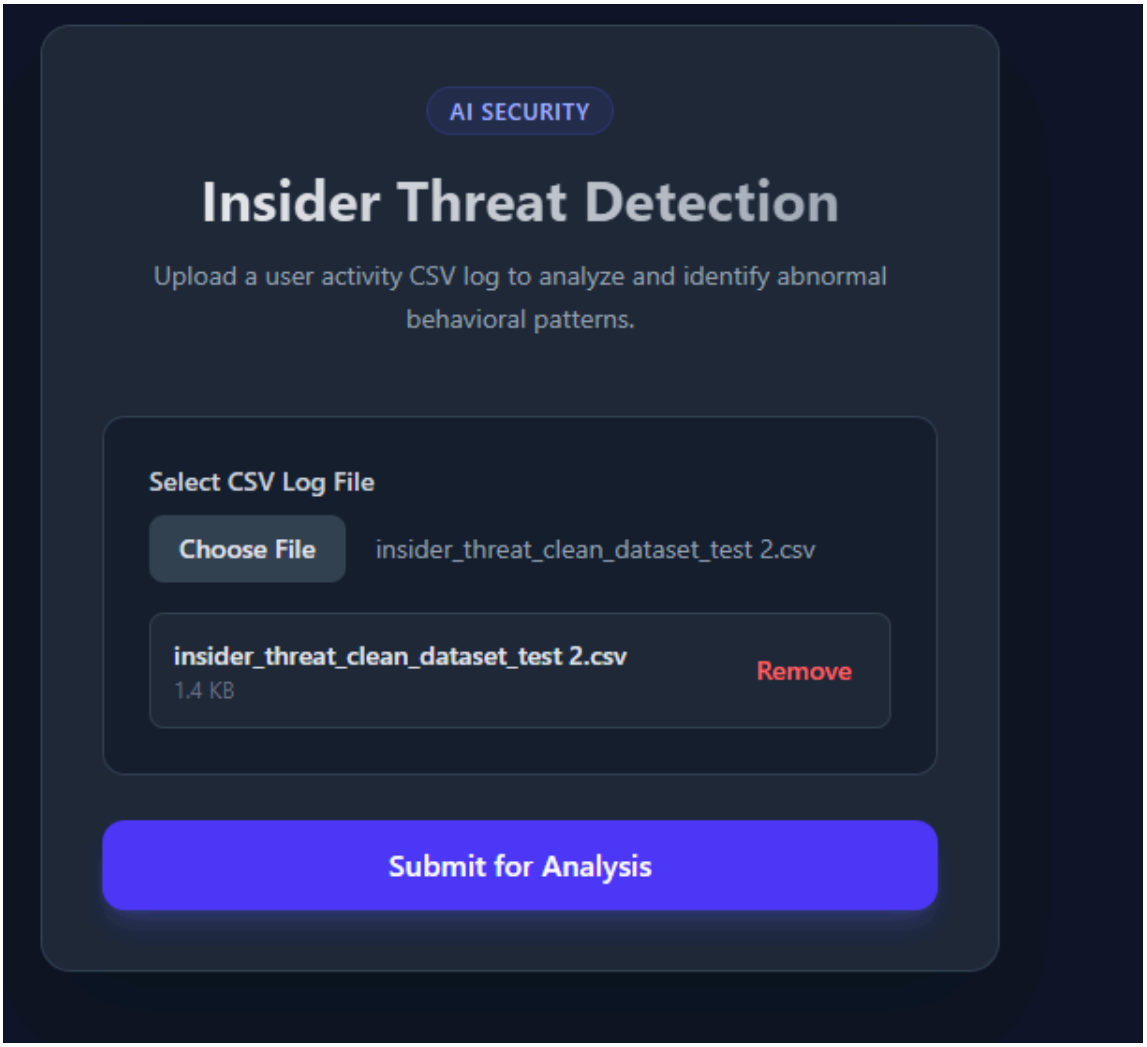
Artefact Dependency Summary

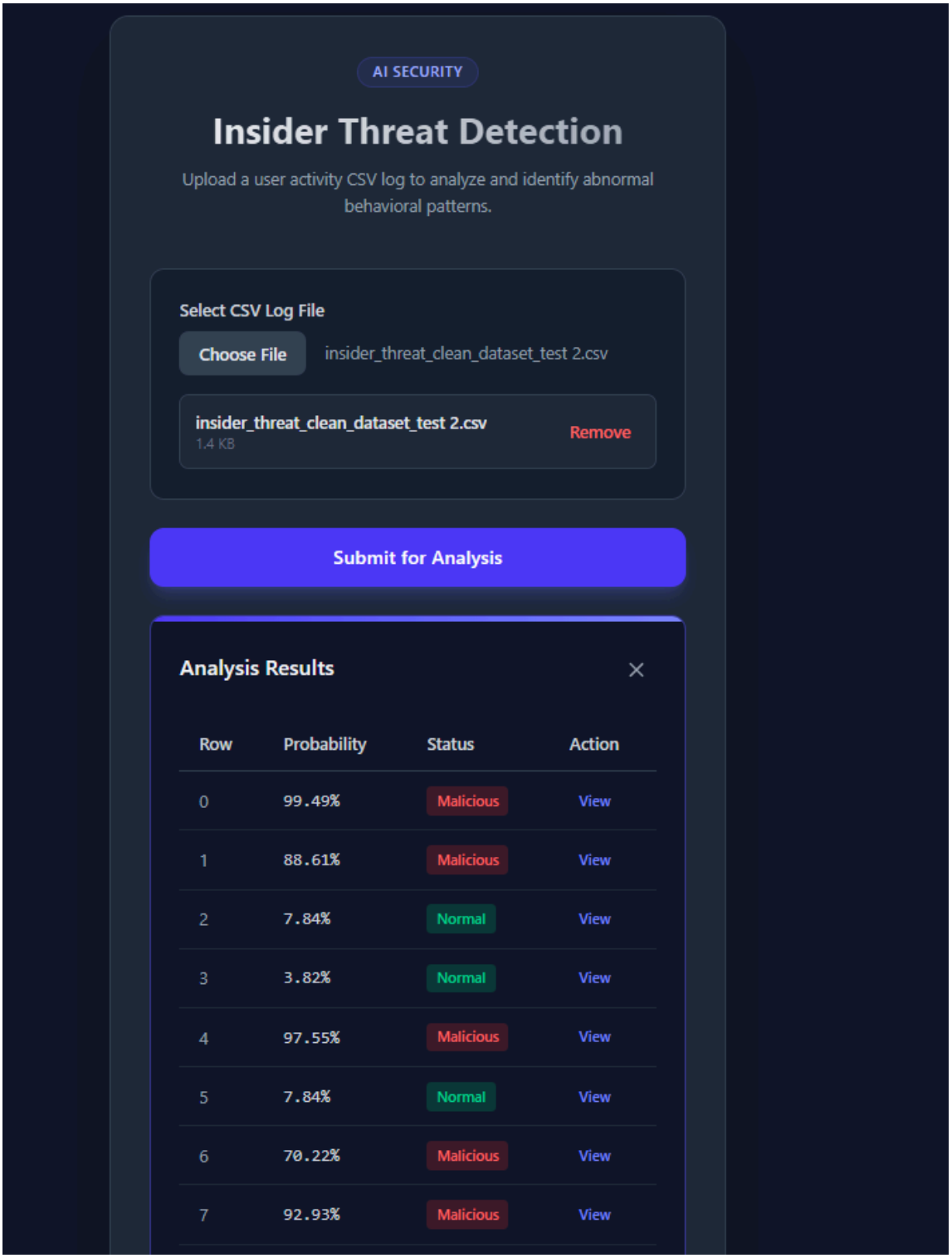
```
Incoming request
      │
      ▼
 scaler.joblib       ← normalises raw input features
      │
      ▼
 model.joblib        ← scikit-learn estimator (primary prediction)
 model_weights.pth   ← PyTorch weights (neural network component)
      │
      ▼
 Prediction output (class + confidence)
```

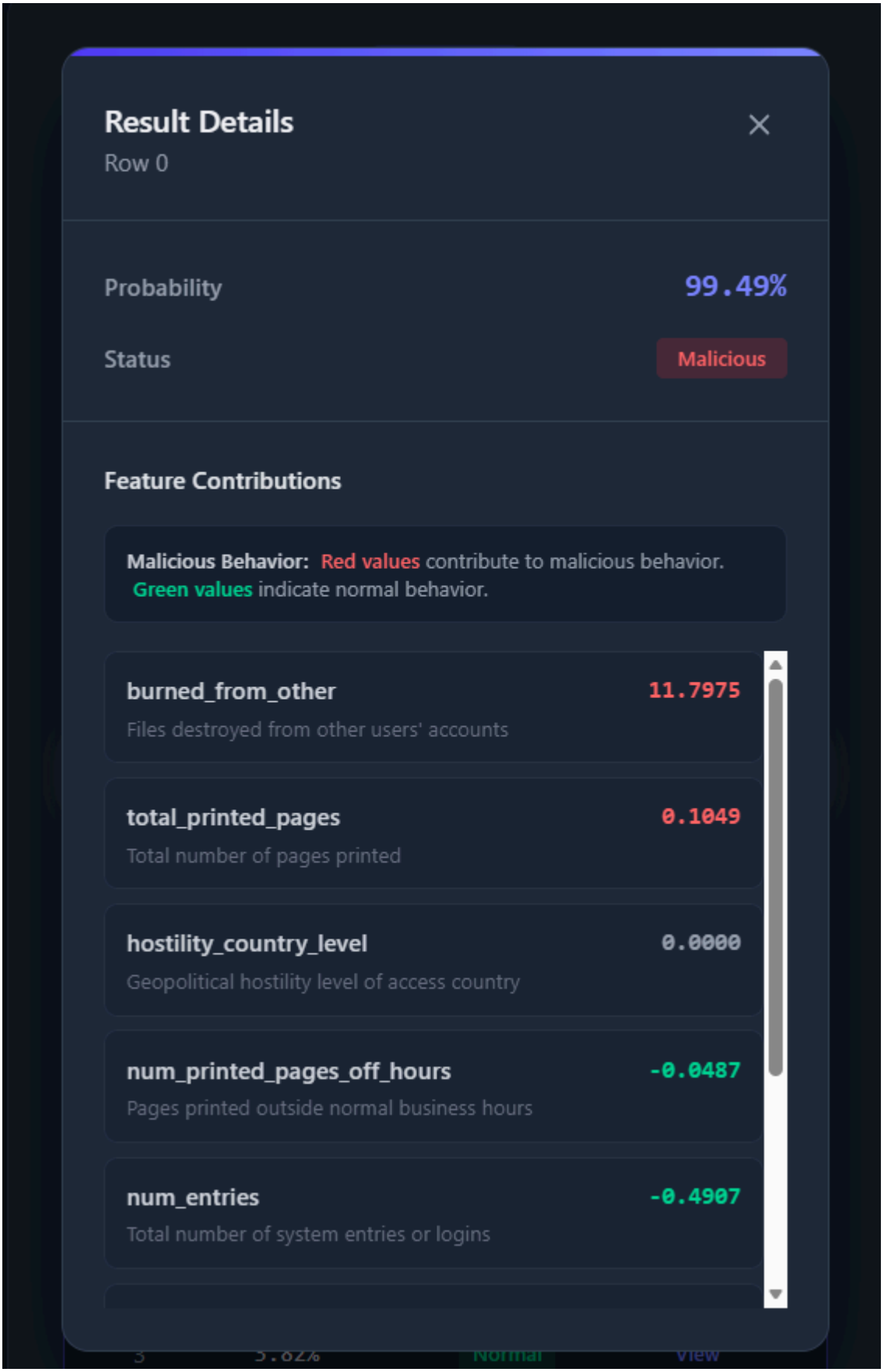
---

7. Screenshots Demonstrating Prototype Functionality


AI SECURITY
Insider Threat Detection
Upload a user activity CSV log to analyze and identify abnormal behavioral patterns.
Select CSV Log File
Choose File
No file chosen
Submit for Analysis


AI SECURITY
Insider Threat Detection
Upload a user activity CSV log to analyze and identify abnormal behavioral patterns.
Select CSV Log File
Choose File
insider_threat_clean_dataset_test 2.csv
insider_threat_clean_dataset_test 2.csv
1.4 KB
Remove
Submit for Analysis


AI SECURITY
Insider Threat Detection
Upload a user activity CSV log to analyze and identify abnormal behavioral patterns.
Select CSV Log File
Choose File
insider_threat_clean_dataset_test 2.csv
insider_threat_clean_dataset_test 2.csv
1.4 KB
Remove
Submit for Analysis
Analysis Results
Row
Probability
Status
Action
0
99.49%
Malicious
View
1
88.61%
Malicious
View
2
7.84%
Normal
View
3
3.82%
Normal
View
4
97.55%
Malicious
View
5
7.84%
Normal
View
6
70.22%
Malicious
View
7
92.93%
Malicious
View


Result Details
Row 0
Probability
99.49%
Status
Malicious
Feature Contributions
Malicious Behavior: Red values contribute to malicious behavior.
Green values indicate normal behavior.
burned_from_other
11.7975
Files destroyed from other users' accounts
total_printed_pages
0.1049
Total number of pages printed
hostility_country_level
0.0000
Geopolitical hostility level of access country
num_printed_pages_off_hours
-0.0487
Pages printed outside normal business hours
num_entries
-0.4907
Total number of system entries or logins